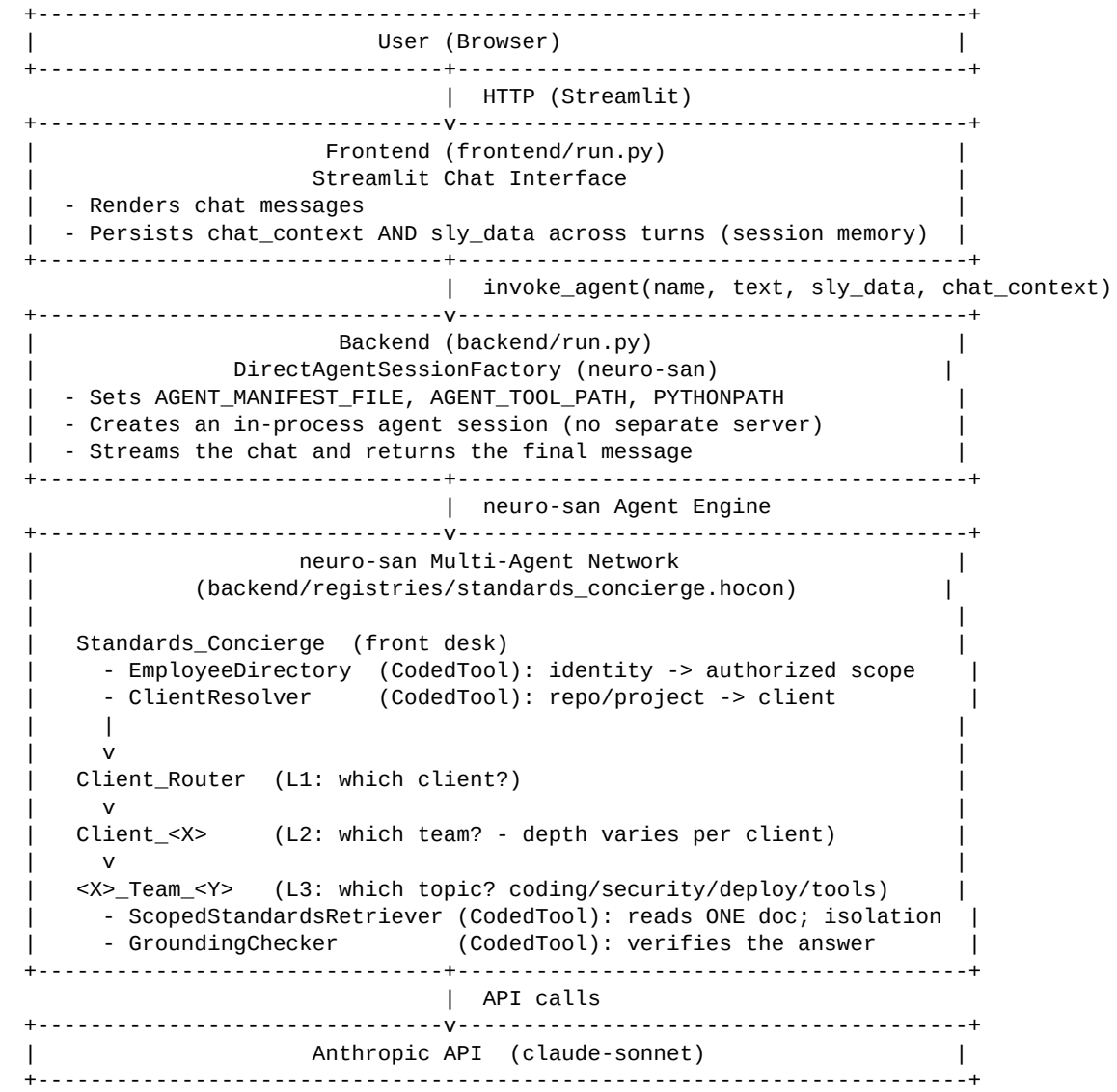
Architecture

Overview

This project is an AI-powered multi-client engineering-standards assistant built on Cognizant's neuro-san framework. It accepts a developer's plain-English question and routes it through three real levels - client -> team -> topic - then answers using only that client's standards. An identity layer restricts each developer to the client they are authorized for.

The system is a Streamlit web frontend, a Python backend, and a multi-agent network orchestrated by neuro-san.

System Architecture Diagram



Component Breakdown

1. Frontend - frontend/run.py

A Streamlit chat interface that renders the conversation, calls the backend's invoke_agent(), and persists both chat_context and sly_data in session state so identity and current client are remembered across turns.

2. Backend - backend/run.py

A thin module that bootstraps env vars (AGENT_MANIFEST_FILE, AGENT_TOOL_PATH, PYTHONPATH), loads .env, and exposes invoke_agent(agent_name, user_text, sly_data, chat_context) using neuro-san's in-process DirectAgentSessionFactory.

3. Agent Registry - backend/registries/

File	Purpose
manifest.hocon	Lists the active agent network
standards_concierge.hocon	Defines the 29 agents/tools and their instructions
aaosa.hocon	AAOSA substitution variables (aaosa_instructions, aaosa_call)
llm_config.hocon	Shared LLM configuration (Anthropic claude-sonnet)

4. Multi-Agent Network - standards_concierge.hocon

Three genuine routing levels (client, team, topic) plus four deterministic coded tools. Team depth is uneven by design (Banking A-D ... Telecom A) so routing is dynamic.

5. CodedTools - backend/coded_tools/standards_concierge/

Tool	Role
EmployeeDirectory	Maps a developer's name to the one client+team they may access
ClientResolver	Resolves a repo/project to its client and remembers it in sly_data
ScopedStandardsRetriever	Reads exactly one client/team/topic doc; enforces isolation + access in code
GroundingChecker	Flags any answer claim not supported by the retrieved text

---

Agentic System Highlights

- Hierarchical AAOSA: three real routing levels; every delegating agent negotiates the inquiry via aaosa_instructions / aaosa_call.
- sly_data session memory: current client and identity are carried out-of-band and reused across turns - never re-typed into the prompt.
- Scoped-retrieval CodedTools: the retriever builds a path only from the requested client and verifies it stays inside that client's folder - cross-client reads are structurally impossible.
- Code-enforced access control: identity limits scope in Python, so the model cannot be prompted into crossing a boundary.
- Grounding evaluation loop: every answer is checked against the source; if no rule exists, the system escalates instead of inventing one.

Data Flow

```
User: "Hi, I'm amirthap. Can I use a GPL library?"
  -> Frontend calls invoke_agent('standards_concierge', text, sly_data, chat_context)
  -> Backend creates a DirectAgentSession for 'standards_concierge'
  -> EmployeeDirectory: amirthap -> authorized Client_Banking / Team_A (into sly_data)
  -> Client_Router -> Client_Banking (L2) -> Banking_Team_A (L3)
  -> ScopedStandardsRetriever(client=Banking, team=Team_A, topic=security)
  -> GroundingChecker verifies the drafted answer against the doc
  -> Final response (text + chat_context + sly_data) returned to the frontend
```

Technology Stack

Layer	Technology
Agent Framework	neuro-san 0.6.71
LLM	Anthropic claude-sonnet
Frontend	Streamlit 1.56.0
Agent Config	HOCON
Custom Tools	Python 3 (CodedTool interface)
Retrieval	Local Markdown files (no vector DB - reliability for a live demo)